

Unity / C# / 게임 클라이언트 개발자

정우태 Unity 클라이언트 개발자

Unity와 C#으로 모바일, WebGL, VR 프로젝트를 만들었습니다. 대표 작업은 Google Play 출시작 Ghost March, AI 협업 구조를 설계한 Match3LogueLike, VR 팀 프로젝트 IronSarcophagus입니다.

2

공개 배포 프로젝트

5

대표 Unity 프로젝트

67fps

DOTS 4096 enemies 기준

34

Match3 테스트 파일

Web

nulma.github.io

GitHub

github.com/NulMa

Email

jungwt524@gmail.com

목차

핵심 사례는 Ghost March와 Match3LogueLike입니다. Ghost March는 Google Play 출시와 성능 재설계, Match3LogueLike는 AI 협업을 위한 구조 설계와 검증 루프를 담고 있습니다.

01

프로젝트 맵

Ghost March, Match3LogueLike, IronSarcophagus, LostCone, Cubika.

03

02

Ghost March

Google Play 출시, Legacy 기준본, DOTS 성능 재설계, 운영 문서.

04-05

03

Match3LogueLike

asmdef, EventBus, AI 작업 범위, PlayMode/EditMode 테스트 근거.

06-07

04

IronSarcophagus

VR 전차 협동 게임, 포수 조준, 탄도 계산, 적 AI.

08

05

LostCone

공개 개발, 팬층 피드백, 방향 전환, Input System 문제 해결.

09-10

06

Cubika

itch.io WebGL 배포, 큐브 병합 메커니즘, 모바일 조작 보강.

11

07

기타 구현 / 작업 방식

Dong_Mae, MyTokenMate, 프로젝트별 공통 작업 기준.

12-13

08

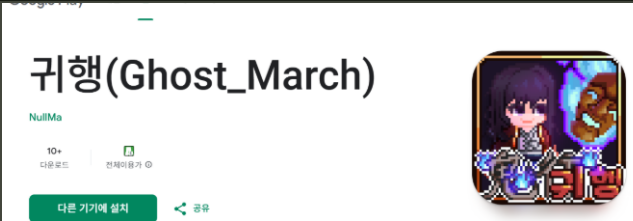
링크

공개 사이트, 프로젝트 영상, GitHub 주소, 연락처.

14

프로젝트 맵

Ghost March는 출시와 운영, Match3LogueLike는 구조 설계, IronSarcophagus는 팀 시스템, LostCone은 공개 개발과 방향 전환, Cubika는 WebGL 배포가 핵심입니다.



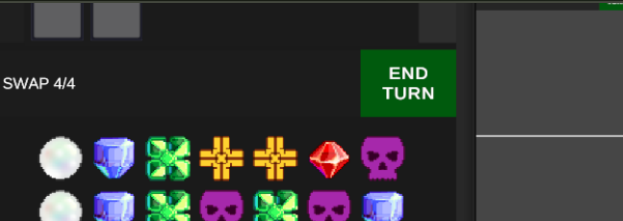
귀행(Ghost_March)

1인 개발 / Google Play 출시

Ghost March

한국 무속 테마의 모바일 survival-like. 출시 후 Legacy 기준본을 보존하고 DOTS 수직 슬라이스로 대량 적 처리 성능을 검증했습니다.

[출시](#) [DOTS](#) [운영](#)



1인 개발 / AI 협업 구조

Match3LogueLike

모바일 portrait 매치3 로그라이크. AI가 구조를 깨지 않도록 asmdef, EventBus, 테스트, 작업 lane을 먼저 설계했습니다.

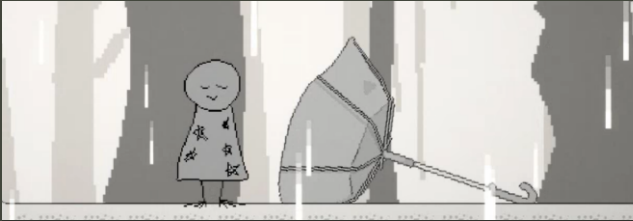
[asmdef](#) [EventBus](#) [테스트](#)



2인 팀 / VR

IronSarcophagus

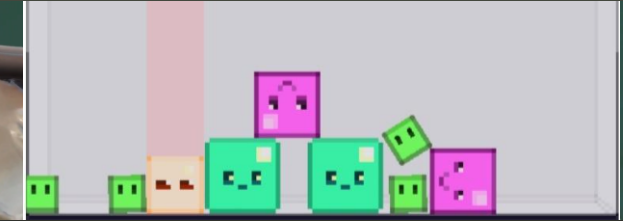
VR 전차 협동 게임. 포수 조준, 탄도 계산, 피해 판정, 적 AI 재배치 흐름을 담당했습니다.



공개 개발 / 팬게임

LostCone

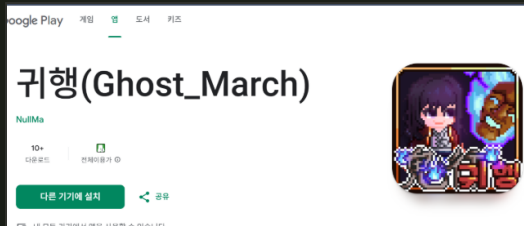
팬층 피드백을 받으며 방향을 바꾼 플랫폼어. Input System 간헐 입력 중단을 환경 상태까지 추적했습니다.



1인 개발 / WebGL

Cubika

정사각형 큐브 병합 퍼즐. itch.io WebGL 배포 후 모바일 조작과 game-over feedback을 보강했습니다.



확인 링크

Play	Google Play
Video	youtu.be/E25iW69OW6k
GitHub	github.com/NulMa/Ghost_March

Ghost March: 출시작을 기준으로 남기고 다시 설계

한국 무속 테마의 모바일 survival-like 프로젝트입니다. 처음 목표는 장르 핵심 루프를 끝까지 완성해 Play Store에 올리는 것이었고, 출시 이후에는 구조적 병목을 분리해 대량 적 처리 성능을 재검증하는 방향으로 이어갔습니다.

개발 목적

작은 범위라도 완성, 수상, 출시까지 가져가며 모바일 클라이언트의 전체 루프를 경험.

핵심 구성

웨이브, 경험치/레벨업, 무기, 특수기 커맨드, 보스, 저장, 일시정지, UI feedback.

차별화 판단

리듬게임/격투게임의 커맨드 입력을 궁극기 게이지 수급과 연결.

운영 확장

AAB release, Billing, App Update, localization, AdMob 문서화.

구조상 문제 인식

- GameManager가 상태, 입력, UI, 저장, 일시정지까지 넓게 소유해 변경 영향 범위가 커졌습니다.
- Weapon.Update()의 switch 기반 분기는 무기 추가 시 조건 분기가 계속 늘어나는 형태였습니다.
- PoolManager prefab 배열 순서가 런타임 id 계약처럼 쓰여 씬과 코드 결합이 강했습니다.

재설계 방향

- Legacy MonoBehaviour 구조는 출시 기준으로 보존해 비교 대상을 남겼습니다.
- Enemy, Bullet, Drop, Wave, Damage Event를 DOTS 수직 슬라이스로 분리했습니다.
- MonoBehaviour는 입력, UI, 외부 서비스, 씬 부트스트랩 쪽으로 제한했습니다.

775 commits

출시 이후 DOTS, 운영 문서, 서비스 기능까지 이어진 작업량.

195 C# files

플레이 루프, UI, 저장, 운영 기능이 분리된 실제 프로젝트 규모.

72 docs

release, IAP, AdMob, App Update, AI 협업 규칙을 문서화.

민감 정보 제외

keystore, real ad id, 인증서 세부값은 PDF에 노출하지 않음.

Ghost March: 성능과 운영에서 남긴 근거

출시 경험만 남긴 것이 아니라, 이후 문제를 측정하고 구조를 바꿔 어떤 차이가 나는지 기록했습니다. 수치는 “구조 변경이 왜 필요했는가”를 설명하는 근거로 사용했습니다.

9fps

Legacy 구조에서 1500 enemies stress scene 기준. 플레이 루프가 유지되지 않는 병목을 먼저 확인했습니다.

67fps

DOTS 수직 슬라이스로 이동/피해/렌더링을 분리한 뒤 4096 enemies 기준으로 다시 측정했습니다.

109.5ms → 14.92ms

CPU frame time을 기본본과 실험본으로 나눠 비교해, 구조 변경의 효과가 숫자로 남는지 확인했습니다.

691 → 4

렌더링 batch 폭증을 instancing 적용 범위로 좁혀 GPU/CPU 양쪽 비용을 같이 줄였습니다.

트러블슈팅 1: 적 이동 보정

적이 과밀집될 때 충돌을 무시하고 무리 안쪽으로 파고드는 문제가 있었습니다. Flow Field만으로는 경로가 난잡해져, 목표 방향은 유지하되 밀집 구간에서만 보정 벡터를 더하는 방식으로 분리했습니다.

트러블슈팅 2: 렌더링 선택

RenderMeshIndirect 도입 후 URP 2D와의 궁합으로 GPU 비용이 오히려 커졌습니다. 목표가 “최신 API 사용”이 아니라 안정적인 결과였기 때문에 측정값이 더 안정적인 DrawMeshInstanced 쪽을 유지했습니다.

트러블슈팅 3: UI 비용

UI Toolkit label.text를 매 프레임 갱신하며 패널 전체 re-render가 발생했습니다. 점수와 상태값이 실제로 바뀐 경우에만 label을 갱신하게 바꾸고, dirty 빈도를 줄여 GPU/CPU 시간을 함께 낮췄습니다.

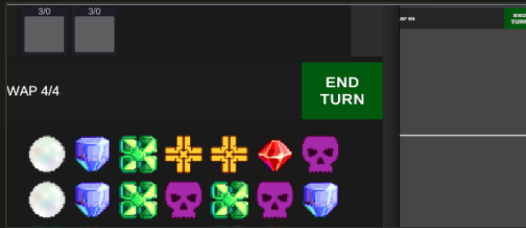
운영 처리 범위

AAB manifest/version/signing, Billing permission, App Update runtime class, localization workflow를 release checklist로 관리했습니다. 출시 이후에도 바뀌는 값과 민감 정보는 코드 설명에서 분리했습니다.

판단 기준

성능 실험은 “무조건 DOTS 전환”이 아니라, 기존 출시본을 비교 기준으로 보존한 뒤 병목이 뚜렷한 부분만 수직 슬라이스로 분리했습니다. 결과가 나쁘면 되돌릴 수 있는 범위로 실험했습니다.

얻은 것: 출시한 코드를 “끝난 코드”로 두지 않고, 기본본과 실험본을 분리해 비교해야 구조 변경의 효과와 위험을 동시에 볼 수 있다는 점을 확인했습니다. 성능 개선은 기술 교체 자체보다 기존 장면, 측정 항목, 되돌릴 수 있는 변경 범위를 먼저 정하는 일이 중요했습니다.



확인 링크

Video youtu.be/mGgBsoj-2JA

GitHub github.com/NulMa/Match3LogueLike

Match3LogueLike: AI가 건드릴 수 있는 경계를 먼저 설계

모바일 portrait 매치3 로그라이크입니다. 개발 목적은 단순히 AI로 코드를 많이 만드는 것이 아니라, AI가 작업해도 구조가 깨지지 않도록 모듈 경계, 금지 규칙, 테스트 evidence를 먼저 만드는 것이었습니다.

구성

Match3.Data: SO / interface / enum, MonoBehaviour 금지

Match3.Core: EventBus, 공통 런타임 규칙

Board / Combat / Roguelike / Meta

UI / Ads / Tests

판단 근거

- Board와 Combat은 직접 참조하지 않고 EventBus로만 통신하게 했습니다.
- 원격 JSON, 로컬 캐시, ScriptableObject 기본값 순서로 데이터 fallback을 정했습니다.
- 생성 코드가 영향을 줄 수 있는 범위를 asmdef와 test assembly로 제한했습니다.

11 asmdefs

Data/Core/Board/Combat/UI/Ads/Tests 등으로 책임 분리.

34 test files

PlayMode/EditMode test로 회귀 탐지.

8 agents

프로젝트 모듈과 1:1에 가까운 작업 역할.

6 hooks

Find, Update 할당, Data MonoBehaviour 등 위험 패턴 감지.

Match3LogueLike: 검증 루프와 남은 리스크를 분리

기능 테스트가 통과해도 실제 모바일 화면에서 읽히지 않을 수 있습니다. 그래서 문서, 테스트 근거, 화면 QA를 다른 레이어로 분리했습니다.

작업 문서

GDD, 구조 문서, asmdef 구조, AI 협업 계획, 멀티 에이전트 운영 문서를 먼저 작성해 작업자가 바뀌어도 기준을 잃지 않게 했습니다. 문서는 설명용 산출물이 아니라 생성 코드가 따라야 하는 경계로 사용했습니다.

테스트 근거

RewardOverlayTests 10/10, SaveManagerPhase1SmokeTests 4/4 등 PlayMode/EditMode 결과를 따로 남겼습니다. 기능 단위 테스트와 통합 smoke test를 나눠, 실패했을 때 어느 레이어가 깨졌는지 먼저 보게 했습니다.

화면 QA

Simulator 화면에서의 가독성, 터치 영역, 보상 overlay 상태를 별도 기록으로 남겨 “테스트 통과”와 “사용 가능”을 분리했습니다. 모바일 portrait에서는 배치가 맞아도 손가락 영역과 텍스트 크기 문제가 따로 생겼습니다.

1. 설계

GDD, 데이터 우선순위, 모듈 경계, 금지 규칙을 먼저 정합니다. 여기서 벗어난 제안은 구현 전에 걸러냅니다.

2. 생성

AI 작업은 정해진 lane과 asmdef 안에서만 진행하게 합니다. Data layer에는 MonoBehaviour 참조를 허용하지 않았습니다.

3. 검증

테스트, architecture scan, merge preview, git diff check를 분리합니다. 통과 기준이 하나로 합쳐지지 않게 했습니다.

4. 리뷰

스크린샷과 화면 QA로 실제 플레이 화면의 문제를 다시 봅니다. 보상창, 터치 영역, 작은 텍스트를 따로 확인했습니다.

남은 리스크 처리

미완성 기능을 완성처럼 쓰지 않기 위해 구현된 범위, 테스트된 범위, 문서상 계획을 분리했습니다. 이 프로젝트의 핵심은 “현재 어디까지 안전한가”를 보이게 만드는 것이었습니다.

다음 프로젝트로 옮긴 기준

AI 협업은 속도보다 변경 통제 비용이 더 큰 문제가 될 수 있습니다. 그래서 이후 작업에서도 asmdef, namespace, interface, 테스트 evidence를 먼저 세우고 기능을 붙이는 순서를 유지합니다.

얻은 것: AI 협업에서 중요한 것은 “AI가 할 수 있는 일”보다 “AI가 하면 안 되는 일을 막는 구조”였습니다. 생성 결과를 믿기 전에 경계, 금지 규칙, 검증 결과를 서로 다른 문서와 테스트로 남겨야 했습니다.



확인 링크

Video youtu.be/l2X7-3QnWks

GitHub [Bit-Unity15th-VRProjects/IronSarcophagus](https://github.com/Bit-Unity15th-VRProjects/IronSarcophagus)

IronSarcophagus: 팀 프로젝트에서 맡은 시스템 경계

“협동의 불편함이 재미가 된다”는 컨셉의 VR 전차 시뮬레이션입니다. 제 담당은 포수 조준, 탄도 계산, 적 AI, 전차 상태 연결이며 팀원 시스템과 맞닿는 지점은 이벤트와 인터페이스로 나뉘었습니다.

본인 담당

- 네트워크 기반 구조와 게임플레이 모드 정리.
- 포수 VR 조준, BallisticCalc, FireSystem 구현.
- 적 AI의 노출도 판정과 NavMesh 기반 재배치 후보 탐색.
- TankStatus와 피해 판정 흐름 연결.

시스템 흐름

AimController: 포수 조준 입력

BallisticCalc: 포물선 / TimeOfFlight

FireSystem: 발사체 / Linecast / impact event

TankStatus: 부위 HP / 관통 / 파괴 상태

5-point exposure

플레이어 주요 포인트를 Linecast해 노출도를 계산.

20% 기준

노출이 낮으면 차폐 상태로 판단.

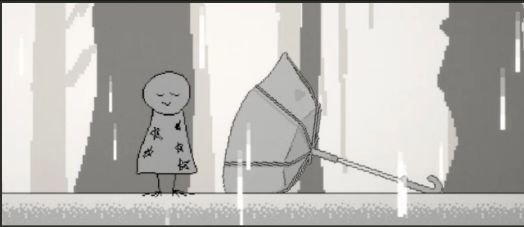
12방향 후보

NavMesh와 시야 확보 조건으로 재배치 위치 탐색.

7 asmdefs

팀원이 같은 파일을 만지지 않도록 경계 분리.

얻은 것: 팀 프로젝트에서는 “많이 구현했다”보다 “어디까지가 내 책임인지, 어떤 이벤트와 인터페이스로 연결되는지”를 먼저 정리해야 충돌이 줄어듭니다.



확인 링크

Video	youtu.be/6QyYSIKdJPw
Devlog	공개 개발 기록
GitHub	github.com/NulMa/LostCone

LostCone: 공개 개발에서 방향을 바꾼 팬게임

팬층을 대상으로 한 플랫폼 프로젝트입니다. 처음에는 퍼즐 요소가 있는 메트로배니아에 가깝게 생각했지만, 실제 유저층이 복잡한 게임 문법보다 세계관과 이스터에그를 기대한다는 판단으로 방향을 바꿨습니다.

개발일지 50회

커뮤니티에 진행 상황을 공개하며 반응을 확인.

플레이테스트 3회

구글 폼과 직접 플레이 반응으로 조작/난이도 확인.

Aseprite

리소스를 직접 만들며 팬게임의 톤을 맞추.

DOTween

프롤로그, fade, 아이템 이동 연출에 활용.

목적

좋아하는 IP의 분위기를 플레이 가능한 형태로 만들고, 실제 팬층 반응을 보며 완성 방향을 조정하는 것.

구성

플랫폼 이동, 컷씬, 아이템, 레벨 진행, 감성 연출, 저장 기능, 플레이테스트 배포 흐름.

판단

난이도 높은 메트로배니아보다 팬층이 찾고 즐길 수 있는 힐링/탐색 방향이 더 맞다고 판단.

1. 공개 기록

개발일지로 진행 상황을 공개하고, 팬층이 어떤 장면에 반응하는지 확인했습니다.

2. 테스트

플레이테스트와 설문으로 조작, 난이도, 길찾기, 연출 이해도를 수집했습니다.

3. 방향 전환

복잡한 퍼즐보다 이스터에그와 분위기 탐색을 강화하는 쪽으로 조정했습니다.

4. 구현 반영

컷씬, fade, 아이템 이동, 저장 기능을 팬게임의 감성 유지에 맞춰 다듬었습니다.

얻은 것: 내가 만들고 싶은 구조보다 실제 플레이할 사람의 이해도와 기대를 먼저 봐야 한다는 점을 공개 개발 과정에서 확인했습니다.

LostCone: 간헐적 입력 중단을 코드 밖 상태까지 추적

같은 코드와 같은 커밋에서도 키보드 입력이 중단되는 문제였습니다. 코드 변경만 반복해서는 원인을 잡을 수 없었고, 플레이 모드의 Input System 상태를 직접 확인하며 원인을 분리했습니다.

1. 증상

플레이 중 키보드 입력이 간헐적으로 끊기고, 같은 커밋에서도 재현 조건이 일정하지 않았습니다. 저장 데이터나 씬 전환 문제로 보였습니다.

2. 실패한 시도

ActionMap 강제 활성화, polling 전환, sceneLoaded, Domain Reload, Library 삭제, git checkout을 차례로 확인했지만 증상이 사라지지 않았습니다.

3. 원인

터치스크린 노트북에서 Input System이 Control Scheme을 Touch로 auto-switch하고 있었습니다. 코드 diff가 아니라 실행 환경 상태 문제였습니다.

4. 해결

Keyboard&Mouse scheme 강제, prefab default scheme 변경, auto-switch 해제로 입력 장치를 고정했습니다. 이후 같은 장면에서 입력이 유지되는지 다시 확인했습니다.

왜 오래 걸렸는가

코드상 입력 처리만 보면 ActionMap, Update timing, scene load 순서 모두 의심할 수 있었습니다. 하지만 문제는 코드가 아니라 현재 연결된 장치와 Control Scheme 선택 상태였습니다. 코드만 되돌리면 해결될 것이라는 가정이 오히려 추적을 늦췄습니다.

다음 작업에서 바뀐 점

간헐적 문제를 만나면 먼저 런타임 Inspector, 장치 상태, project setting, prefab default 값을 같이 확인합니다. “코드가 같으면 동작도 같다”는 가정을 줄이고, 엔진 설정과 디바이스 상태를 체크리스트에 넣었습니다.

확인한 코드

InputAction, ActionMap 전환, Update polling, sceneLoaded 연결 순서를 확인했습니다.

확인한 엔진 상태

PlayerInput의 current scheme, prefab default scheme, project input settings를 비교했습니다.

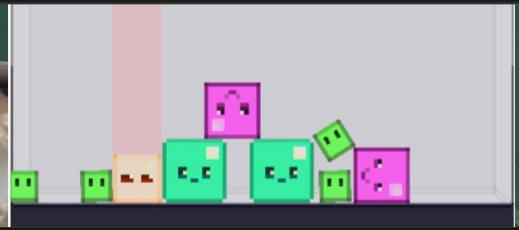
확인한 환경

터치스크린 장치가 연결된 노트북에서 Touch scheme으로 전환되는 상황을 분리했습니다.

남긴 기준

재현이 불안정한 버그는 코드 변경 전에 런타임 상태를 먼저 기록합니다.

얻은 것: 재현이 흔들리는 버그는 코드 diff만 볼 것이 아니라 엔진 상태, 입력 장치, 실행 환경까지 관찰해야 합니다. 같은 소스라도 런타임에서 선택된 장치와 scheme이 다르면 완전히 다른 결과가 나올 수 있었습니다.



확인 링크

Play	nullma.itch.io/cubika
Shorts	youtube.com/shorts/A6-j0cwDtW4
GitHub	github.com/NulMa/Cubika

Cubika: 작게 완성하고 WebGL로 배포

수박게임류 병합 퍼즐에서 출발했지만, 원형이 아니라 정사각형 큐브의 회전, 낙하, 밀치기를 사용해 플레이 감각을 바꾼 프로젝트입니다. 짧은 기간에 완성해 itch.io에 WebGL로 올리고, 이후 모바일 조작과 수익화 scaffold를 보강했습니다.

WebGL 배포

itch.io에서 바로 플레이 가능한 형태로 공개.

밀치기 메커닉

떨어진 큐브의 첫 유효 충돌에만 밀치기 의도를 적용.

3초 grace

over-line cube가 3초 지속될 때만 game over.

광고/IAP 구조

Unity Ads, remove-ads product scaffold 추가.

개발 목적

작은 게임을 끝까지 완성하고, 빌드-업로드-피드백 보강까지 경험하는 것.

문제 해결

game over가 너무 즉시 발생하면 납득하기 어려워 countdown feedback과 grace period를 추가했습니다.

얻은 것

완성 후의 작업은 새 기능보다 입력, feedback, 일시정지, 확인 팝업 같은 사용성 보강이 중요하다는 점을 확인했습니다.

메커닉 판단

원형 병합의 예측 가능한 흐름을 그대로 복제하지 않고, 사각형 큐브의 회전과 밀치기로 불안정하지만 읽을 수 있는 물리 퍼즐을 만들었습니다.

출시 후 보강

MainMenu, pause popup, confirmation dialog, mobile controls, Unity Ads/IAP scaffold를 추가해 단순 업로드 이후의 운영 항목을 확인했습니다.

기타 구현 경험

Dong_Mae는 레벨 기믹과 연출 구현 폭을 보여주고, MyTokenMate는 개발 중 반복해서 생긴 AI 사용량 확인 문제를 local-only 도구로 푼 사례입니다.

Dong_Mae

메트로배니아 팀 프로젝트에서 레벨 기믹과 연출 구현 폭을 확보했습니다. 타일맵, 아이템, 문, 로프, 점프패드, 사다리, 이동 플랫폼, 함정, 수중, 파괴 가능한 벽, 카메라 전환, fade, 보스 레터박스 연출을 다뤘습니다.

MyTokenMate

AI coding tool 사용량을 작업 중 확인하기 위한 Electron 오버레이입니다. Claude Code, Codex, Gemini CLI, Copilot의 로컬 로그를 파싱해 API key나 네트워크 요청 없이 상태를 보여주는 방향으로 구현했습니다.

콘텐츠 구현

LostCone, Cubika, Dong_Mae에서 조작, 레벨, 연출, feedback을 직접 맞추며 장르별 사용감을 조정했습니다.

도구 제작

MyTokenMate는 게임 프로젝트는 아니지만, 실제 개발 중 겪는 AI 사용량 확인 문제를 local-only 도구로 풀려는 시도입니다.

주의한 점

대시보드처럼 외부 도구를 가져와 적용한 사례는 직접 제작 프로젝트처럼 쓰지 않고 작업 방식 근거로만 분리했습니다.

Dong_Mae

레벨 기믹, 보스 패턴, 카메라/페이드 연출 구현 폭.

MyTokenMate

Claude/Codex/Gemini/Copilot local log parsing 실험.

분리 기준

대표 프로젝트와 보조 근거를 같은 비중으로 섞지 않음.

다음 조건

빌드 검증, parser fixture test, 실제 screenshot 확보 시 독립 페이지 후보.

작업 방식

프로젝트마다 형태는 달라도 반복되는 기준은 같습니다. 먼저 목적과 경계를 정하고, 구현 후에는 테스트/영상/실제 화면/운영 문서로 결과를 다시 확인합니다.

1. 목적 정의

게임을 왜 만들고, 어떤 결과가 완료 기준인지 먼저 정합니다. Ghost March는 출시, Match3는 구조 설계, LostCone은 팬층 피드백이 기준이었습니다.

2. 구조 분리

asmdef, EventBus, interface, namespace, 문서로 변경 영향 범위를 작게 만듭니다.

3. 검증 분리

테스트 통과, 성능 수치, 영상, visual QA, 플레이테스트를 서로 다른 근거로 분리합니다.

4. 회고 반영

문제를 해결하고 끝내지 않고, 다음 프로젝트의 설계 기준으로 옮깁니다.

출시

AAB, store, WebGL upload, 링크 공개.

성능

기준본과 실험본을 분리해 수치 비교.

협업

팀원/AI 모두 작업 경계와 검증 형식을 먼저 정의.

트러블슈팅

코드, 엔진 상태, 입력 장치, 실행 환경을 함께 확인.

프로젝트별 적용

- Ghost March: 출시 기준본을 보존하고 DOTS 실험본과 수치 비교.
- Match3LogueLike: AI 작업 전에 모듈 경계와 금지 규칙을 정의.
- IronSarcophagus: 본인 담당 시스템과 팀원 담당 시스템을 문서로 분리.

트러블슈팅 기준

- LostCone: 입력 코드를 고치기 전에 Control Scheme과 장치 상태를 확인.
- Cubika: 즉시 game over 대신 feedback과 grace period로 납득 가능한 실패 상태 구성.
- Ghost March: 최신 렌더링 API보다 프로젝트 렌더 파이프라인과 맞는 선택을 우선.

링크 / 연락처

항목	유형	주소	비고
Web Portfolio	Site	https://nulma.github.io/	공개 포트폴리오
GitHub	Profile	https://github.com/NulMa	코드/프로필
Ghost March	Play	play.google.com/store/apps/details?id=com.NullMa.GhostMarch	Google Play
Ghost March	Repo	github.com/NulMa/Ghost_March	GitHub 주소
Match3LogueLike	Repo	github.com/NulMa/Match3LogueLike	GitHub 주소
IronSarcophagus	Repo	github.com/Bit-Unity15th-XRProjects/IronSarcophagus	팀 저장소 주소
LostCone	Repo	github.com/NulMa/LostCone	GitHub 주소
Cubika	Play	nullma.itch.io/cubika	WebGL 배포
Cubika	Repo	github.com/NulMa/Cubika	GitHub 주소
LostCone	Devlog	gall.dcinside.com/.../view/?id=yonezkensi&no=114242	공개 개발 기록
MyTokenMate	Repo	github.com/NulMa/my-token-mate	보조 도구
Email	Contact	jungwt524@gmail.com	연락처